

**Role-based access control (RBAC)** means users only get access to the parts of the system that match their role. In your project, a student, organization officer, OSA administrator, and system administrator do not have the same permissions. This is used so people cannot open, edit, or manage information that is not part of their responsibility.

**Secure login and authentication system** means the system checks first if the user is really allowed to enter. Users must log in using their account credentials, and the system protects passwords through secure methods like hashing. For higher-level users like administrators, extra protection such as MFA may also be added. This helps stop unauthorized users from entering the system.

**Encrypted data transmission (HTTPS)** means the information sent between the user's device and the website is protected while it is traveling through the internet. This prevents attackers from easily reading or stealing sensitive data like usernames, passwords, or submitted records.

**Centralized and secure database storage** means the system keeps important records in one organized database instead of having them scattered in papers or separate files. Since the database is centralized, it is easier to manage, monitor, retrieve, and protect records. Security measures are also applied so the stored data stays safe from unauthorized access or loss.

**Compliance with data privacy and institutional policies** means the system is designed to follow rules and standards about how data should be handled. In your capstone, this includes protecting student information, limiting access to authorized users, collecting only needed data, and supporting policies related to digital governance and privacy. This makes the system not only functional, but also legally and institutionally responsible.

A very easy way to remember them is:

RBAC = who can access what

Authentication = proving who the user is

HTTPS = protecting data while sending it

Secure database = protecting data while storing it

Compliance = following rules and policies

For defense, you can say:

These features were included to ensure that the system is secure, organized, and policy-compliant. RBAC controls user permissions, authentication verifies identity, HTTPS protects transmitted data, centralized secure storage protects records, and compliance ensures that the system respects privacy and institutional regulations.

#### a) Role-Based Access Control

“First, the system uses role-based access control or RBAC. This means users are only allowed to access the functions and records that match their role. Students, organization officers, and OSA administrators do not have the same permissions. This prevents unauthorized actions and improves accountability.”

#### b) Token-based authentication

“Second, the system uses token-based authentication through JWT with HMAC-SHA256. This helps verify the identity of the user securely and prevents token tampering or unauthorized access.”

#### c) Secure communication

“Third, the system uses secure transmission such as HTTPS or TLS so that data being sent between the user and the server is protected.”

#### d) Audit logging

“Fourth, the system records important actions through audit logs. Login attempts, role assignments, service updates, rental transactions, workflow status changes, approvals, and administrative changes are recorded with the user ID, timestamp, and affected record. This helps in traceability, compliance checking, and incident investigation.”

A short defense line:

“So security in our system is not just about login protection. It also includes access limitation, secure communication, and traceable user actions.”

---

#### 4) How to explain Data Privacy

Say this clearly:

“Data privacy focuses on protecting the personal and organizational information stored in the system. Since the platform handles student-related and institutional data, privacy is addressed through privacy-by-design principles.” [OBJ]

Then explain the parts:

a) Controlled access

“Only authorized users can view specific records. Access is role-scoped, so sensitive data is not exposed to everyone.” [OBJ]

b) Data minimization

“The system only collects and uses the information necessary for student organization governance and services.” [OBJ]

c) Retention and handling

“Rental and organizational documents are archived or securely deleted after defined institutional retention periods. This means data is not kept forever without purpose.” [OBJ]

d) Privacy rights and incident response

“The system also includes procedures for data access, correction, complaints, and breach response. If a security incident happens, there are steps for detection, containment, documentation, and notification.” [OBJ]

Strong line for panel:

“So our system does not only secure the data technically, it also handles the data responsibly in line with privacy principles.”

## TECHSTACK

The system uses HTML, CSS, and JavaScript for the frontend to build the user interface and interactive features. For the backend, it uses PHP to handle server-side logic, data processing, authentication, and communication with the database. The database used is MySQL, which stores and manages the system’s records and transactions. The system is run in a XAMPP local development environment, which provides the Apache web server, PHP runtime, and MySQL/MariaDB services needed to develop and test the application locally.

**If you want a more formal version for a thesis or capstone paper, you can use this:**

**The proposed system is developed using HTML, CSS, and JavaScript for the front-end interface, PHP for server-side scripting and backend functionality, and MySQL for database management. To support local development and deployment, XAMPP is used as the development environment, providing Apache as the web server and integrating PHP and MySQL services for seamless system operation.**

It is a `modular monolith` because the system is built as `one whole application`, but its features are `organized into separate modules`.

Why it is `monolithic`:

- It runs under one project/codebase.
- It uses one main backend technology stack: `PHP` in the same application.
- It uses one centralized `MySQL` database.
- Its modules are deployed together, not as separate independent services.

Why it is `modular`:

- The backend is divided into feature-based parts like `auth`, `accounts`, `documents`, `qr-attendance`, `igp`, `printing`, `lockers`, and `profiles`.
- Each module has its own files, endpoints, and responsibilities.
- This separation makes the code easier to maintain and extend.

So it is not just a plain monolith because the code is structured by module, but it is still not microservices because the modules are not independently deployed.

A simple definition you can use:

`A modular monolith is a single application that is organized into separate functional modules within one unified codebase and database.`

For your paper, you can say:

`The system uses a modular monolithic architecture because it is developed and deployed as a single PHP application while organizing backend functionality into separate modules such as authentication, account management, attendance, rentals, documents, printing, and lockers. These modules share the same codebase and centralized MySQL database, but remain logically separated to improve maintainability and clarity`

#### Role-Based Access Control (RBAC) for user permissions

In the system, RBAC is used to separate access between students, organization officers, and OSA staff. Students are limited to features such as viewing announcements, requesting services, tracking rentals, submitting printing requests, and requesting lockers. Organization officers are given access to organization management functions such as posting announcements, reviewing service transactions, handling locker assignments, monitoring printing requests, and managing organization documents. OSA personnel are given higher-level access for reviewing requests, overseeing organizations, and managing approvals. This helps the system keep each module accessible only to the proper user type.

#### Token-Based Authentication (JWT with HMAC-SHA256)

In the system, token-based authentication can be used after login to verify the identity of the user whenever the frontend accesses protected backend APIs. After a student, officer, or OSA user logs in, the server can issue a signed token that is sent with requests to modules such as document submission, attendance processing, rental transactions, printing requests, and profile updates. This allows the backend to confirm that the request is coming from an authenticated user before returning records or allowing actions inside the system.

#### Encrypted data transmission (TLS) and secure session handling

In the system, encrypted transmission is important when users submit login credentials, personal profile information, uploaded documents, and service requests. TLS can protect the communication between the browser and the server while students and staff use the dashboards, submit documents, process rentals, or scan attendance. Secure session handling is also used to maintain a safe logged-in state while users move between modules such as the student dashboard, officer dashboard, and OSA dashboard without exposing their authentication details.

#### Audit logging and activity monitoring

In the system, audit logging can be used to record important actions performed by users across the modules. For example, it can log when a user logs in, when OSA approves or rejects requests, when officers post announcements, when documents are uploaded or reviewed, when attendance is checked in or updated, and when rental, printing, or locker transactions are processed. This gives the system a history of user activity that can be checked when monitoring operations or investigating problems.